the smaller the protected region, the fewer operations are serialized, and the greater the potential for concurrency.

Before we look at some data structure designs, let's have a quick look at some simple guidelines for what to consider when designing for concurrency.

6.1.1. Guidelines for designing data structures for concurrency

As I just mentioned, you have two aspects to consider when designing data structures for concurrent access: ensuring that the accesses are *safe* and *enabling* genuine concurrent access. I covered the basics of how to make the data structure thread-safe back in chapter 3:

- Ensure that no thread can see a state where the invariants of the data structure have been broken by the actions of another thread.
- Take care to avoid race conditions inherent in the interface to the data structure by providing functions for complete operations rather than for operation steps.
- Pay attention to how the data structure behaves in the presence of exceptions to ensure that the invariants are not broken.
- Minimize the opportunities for deadlock when using the data structure by restricting the scope of locks and avoiding nested locks where possible.

Before you think about any of these details, it's also important to think about what constraints you wish to put on the users of the data structure; if one thread is accessing the data structure through a particular function, which functions are safe to call from other threads?

This is actually quite a crucial question to consider. Generally constructors and destructors require exclusive access to the data structure, but it's up to the user to ensure that they're not accessed before construction is complete or after destruction has started. If the data structure supports assignment, `swap()`, or copy construction, then as the designer of the data structure, you need to decide whether these operations are safe to call concurrently with other operations or whether they require the user to ensure exclusive